# A software factory
# that checks an agent's changes.

Today: build, reject and accept a change. Next: build it by voice.

"Build me an assistant
for my next flight."

A checked build.
A rejected change.
A card you can inspect.

# “Will I make my meeting?”

A question that needs more than a flight number.

FLIGHT · ILLUSTRATIVE

## Landing 17:00

Computed estimate
Check the source timestamp



WEATHER · ILLUSTRATIVE

## Light rain

Arrival-time forecast
A forecast is not an observation



MEETING · COMPUTED EXAMPLE

## 30 minutes late

17:00 landing + 90 min transfer
18:00 meeting, same local date

CURRENCY

## Daily reference

Publication date and source
Unavailable when the source fails

EUR / USD

# The screen makes the answer inspectable.

A useful teaser shows both a success and a failure.

01

## Ask in the Inspector

Type the request. Choose a flight. Give a meeting time. Voice failed on the second turn today, so type.

02

## Change one assumption

Increase the transfer allowance. Watch only the timeline change.

03

## Lose one source

Show weather as unavailable. Keep the other cards useful.

# Most of today belongs to your group.

110 minutes building, 15 minutes meeting other builders.

| Demo | Concepts | Groups | Lab A | Meet | Lab B | Share |
|---|---|---|---|---|---|---|
| 10 | 20 | 10 | 55 | 15 | 55 | 15 |

Minutes from start to finish: 180. Start 14:30, networking break 16:05 to 16:20 London time, close 17:30. The guide lets you work at your own pace.

# Two agents have different jobs.

The factory creates the product. The traveler uses the product.

BUILD TIME · TARGET ARCHITECTURE

## You describe a product

Factory voice assistant → build worker → independent checks → released preview

RUN TIME · TRAVEL SCAFFOLD

## You ask about a trip

Travel voice assistant → source tools → validated data → visual cards

The attached scaffold and builder currently run separately. Their automatic release bridge is extension work.

# Each part has a clear responsibility.

The cards and factory controller are application code.

| Part | Its job |
| --- | --- |
| **Rasa Mantle** | Run the conversation, skills and tool constraints |
| **Deepgram** | Turn speech into text and text into speech |
| **OpenAI Agents API** | Run a builder session that survives restarts, inside a managed harness (the service runs the agent’s loop for you) |
| **Factory controller** | Scope work, verify an attempt and decide release |
| **Source adapters + visual UI** | Fetch data and render trustworthy cards |

# The Agents API runs the builder session.

Python’s OpenAI client is how our code reaches the service.

```
from openai import OpenAI

client = OpenAI()
client.beta.agents.sessions.create(
    agent={...},
    environment={...},
    input=task,
    stream=True,
)
```

## Four things to follow

Agent configuration
Execution environment
Session identity
Events and final status

Conceptual excerpt, not a complete runnable script.
The separate `openai-agents` framework is the Agents SDK.

# MCP is the connection to your application.

A tool call needs a contract, a result and a failure path.

01

## Mantle skill

Requests a bounded operation

02

## MCP service

Validates arguments; starts a job

03

## Build worker

Uses the Agents API

04

## Job result

Returns status and evidence

# A brief becomes a testable request.

“Make it better” leaves the builder guessing.

PRODUCT REQUEST

“Show only two currencies, with the reference date. If rates fail, say they are unavailable.”

ACCEPTANCE EXAMPLES

- Four input rates produce two rows.
- The publication date is visible.
- Daily rates never say “live”.
- Source failure produces no invented rate.

# The builder cannot grade its own homework.

An independent verifier checks the proposed artifact.

BUILDER WORKSPACE

## Task + product files

Allowed changes only
Development tests available

attempt

VERIFIER WORKSPACE

## Protected acceptance

Independent result
Task and artifact fingerprints (short codes made from the files)

The builder may see example tests. It never sees or edits the acceptance tests that decide.

# A completed session still needs a decision.

A setup failure is different from a bad product.

01

## ACCEPT

The attempt passed the specified checks. Release checks still follow.

02

## REJECT

A product requirement failed. Change the proposal and try again.

03

## NOT_READY (could not run)

A dependency, account or verifier failed. No product verdict exists.

# Dark factories need an explicit release policy.

People define the standards and own exceptions.

01

## Requirements

People specify acceptable behavior



02

## Build + checks

Automation produces evidence



03

## Release gate

Policy permits or stops a release



04

## Monitor

Failures trigger rollback or escalation

# Your group owns one small change.

Driver, requirements owner, evaluator. Rotate after the break.

01

## Write examples

Choose a visible feature. Predict the failure cases.

02

## Run the factory

Keep the task, attempt and recorded verdict together.

03

## Show the result

Demonstrate a rejection and explain what you released.

# One guide, two routes.

Choose the route your machine can run today.

| Route | Requirements | Outcome |
| --- | --- | --- |
| **A · Travel + code** | Mac for code gates; license; provider keys for live voice | Voice scaffold, builder, rejection, local card integration |
| **B · Factory method** | Python 3.12; no provider accounts | Acceptance, recovery and duplicate suppression |
| **Work together** | One prepared machine per group | Everyone writes examples and evaluates results |

# Lab A: a brief and a first attempt.

55 minutes. Open PARTICIPANT-GUIDE.md.

01

10 min

Brief + failure examples

→

02

15 min

Travel experience or fallback

→

03

25 min

One bounded builder attempt

→

04

5 min

Record evidence

# A source label changes what the answer means.

The same number can have very different authority.

| Card / value | Meaning | What to show |
|---|---|---|
| **Aircraft position** | An observation at a timestamp | Source and observed time |
| **Arrival time** | Computed estimate | Method and uncertainty |
| **Weather** | Forecast for a place and time | Forecast time and provider |
| **Transfer** | Planning assumption | Editable allowance |
| **Currency** | Daily reference rate | Publication date; never live trading |

# Meet another group.

15 minutes. Bring back one failure your tests missed.

# “What would make your assistant confidently wrong?”

Trade an example. Explain how you would detect it. Swap roles when you return.

# Lab B: rejection, integration and handoff.

55 minutes. Finish with evidence you can show.

01

## 20 min

Reject a misleading attempt

02

## 25 min

Integrate and inspect output

03

## 10 min

Prepare a two-minute demo

# Release the exact artifact that passed.

A changed base or changed attempt invalidates earlier evidence.

TESTED ATTEMPT

## Digest A

Task + verifier + results



RELEASE CHECK

## Still digest A?

Same base? Smoke tests pass?



RUNNING PRODUCT

## Verified version

Preview + rollback reference

The full voice factory also needs a start-and-smoke-test release worker.

# Voice has its own acceptance tests.

Passing text tests leaves important behavior untested.

01

## A short reply

Does “yes” become the right confirmed action?

02

## An interruption

Does playback stop while the user changes the request?

03

## A failed source

Does speech explain the problem while the board keeps useful data?

# Your demo starts with the requirement.

Two minutes per group. Show the failure as well as the success.

## 30 s

The request

## 60 s

The rejected case and the result

## 30 s

The next test

Include your mode: LIVE, SCRIPTED or REPLAY. Name any step you did not run.

# The next step is the complete build-to-voice journey.

One spoken brief should lead to a checked, usable preview.

ALREADY REPRESENTED IN THE PACK

## Travel scaffold + separate builder

Source tools, cards, scoped changes, acceptance and local integration.

REMAINING INTEGRATION

## Voice factory + saved jobs + release

Build tool, status, cancellation, exact-artifact launch and voice smoke tests.

# One question to take back to work.

Your answer determines where to automate next.

## “What evidence would let us release this change without reading every line?”

Start with one bounded change. Keep the rejected cases.